BirdMoE: Reducing Communication Costs for Mixture-of-Experts Training Using Load-Aware Bi-random Quantization

Donglei Wu*†‡, Weihao Yang§, Xiangyu Zou§, Jinda Jia‖, Dingwen Tao¶, Wen Xia§, Zhihong Tian*†‡
*Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China
†Guangdong Key Laboratory of Industrial Control System Security, Guangzhou, China
‡Huangpu Research School of Guangzhou University, Guangzhou, China
§School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China
¶Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China
‖Luddy School of Informatics, Computing, and Engineering, Indiana University, Bloomington, IN, USA

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132853

***Abstract*—Mixture-of-Experts (MoE) model parallelism is prevalent in training Large Language Models (e.g., ChatGPT). However, the intensive *all-to-all* collective communication of the MoE layer's intermediate computing results substantially degrades MoE training efficiency. In this paper, we propose BirdMoE, a novel load-aware communication compression technique with Bi-random quantization for MoE training with two core modules. Specifically, BirdMoE employs a lightweight Random Quantization (RQ) with expectation invariance property to efficiently map the floating-point intermediate computing results into integers while maintaining the MoE training quality. Additionally, BirdMoE utilizes a Mixed Precision (MP) strategy to dynamically balance the communication loads among expert nodes, significantly improving all-to-all communication efficiency for the MoE training system. Experiments on four typical MoE training tasks demonstrate that BirdMoE achieves higher 4.06×-10.44× total communication compression ratios and 1.18×-5.27× training speedup compared with the state-of-the-art compression techniques while maintaining the MoE training quality.**

***Index Terms*—Mixture-of-Experts training, communication compression, load balance.**

I. Introduction

The remarkable success of deep learning can be attributed to the combination of large-scale training datasets and increasingly expansive model architectures. For example, in domains like language generation, machine translation, and computer vision, augmenting model capacity significantly boosts model performance when datasets are suitably large [1]–[5]. However, dense neural networks process every sample using all parameters, resulting in escalating training costs. This includes increased demands on memory and computing power, proportionate to the growth in both model and dataset scales [6].

Mixture-of-Experts (MoE), a form of conditional computation technique, has recently proven effective in expanding model capacity with sub-linear cost [7]. Different from directly scaling small models to a large dense model, the MoE model consists of many small feed-forward networks (FFN), which are named *experts*. Training samples are dynamically selected by a trainable gate network and fed into different experts. Because experts are activated sparsely, MoE notably boosts the number of samples trained within the same time, improving model accuracy with lower computational expenses compared to conventional dense models [8]–[10].

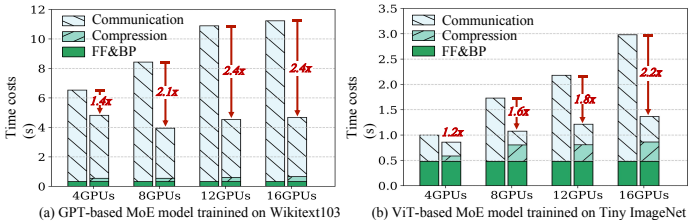

Fig. 1. The runtime breakdown per iteration of naive no-compression MoE training (left bar) and BirdMoE training (right bar).

Communication Bottleneck in MoE: Although the flexible MoE structure makes it more feasible to train a giant model beyond a trillion scale, the training process is still extremely costly. For instance, training a 600-billion-parameter MoE model requires 2048 TPUs and four days [11]. To improve training efficiency, *expert parallelism* is presented to distributedly train MoE models, where experts are partitioned into different workers (e.g., GPU nodes) and each worker processes a different batch of training samples in parallel. In MoE layers, the *intermediate computing results* of each worker are sent to different desired experts during the forward and backward propagation using '*all-to-all*' collective communication [11]. However, the distinct communication characteristics of MoE require the intermediate computing results to be intensively and unevenly transferred across workers in every iteration, making the imbalanced communication overheads a significant bottleneck in MoE training. To closely examine the communication bottleneck, we conduct four typical MoE training tasks on a cloud computing platform (detailed in Section IV). The left bars of Figure 1 indicate that communication times not only take up a large proportion but also increase significantly with the number of expert nodes.

Issues of Existing Solutions: To improve MoE training efficiency, several studies focused on modifying the MoE model architecture [8], [12] or optimizing the training workflow and resource scheduling [9], [10]. However, these works intrude on the standard MoE training workflow, potentially restricting their use in a wide range of practice scenarios. Additionally, as the MoE architecture scales up, the growth of the intensive traffic becomes explosive, making compressing the communication in MoE training a promising direction.

Recently, communication compression techniques, such as sparsification and quantization, have been successfully employed to reduce gradient volume in distributed learning due to the favorable compression ratios and scalability [13]–[20]. However, we noticed that the aforementioned distinct communication characteristics of MoE introduce three specific compression challenges to existing communication compression techniques: ❶ *Overhead Amplification Issue*: Forward and backward propagation of expert parallelism incurs intensive communication across multiple nodes demanding hundreds of compressions per iteration, dramatically amplifying the compression overhead up to 89.6% (as see Figure 2 (a)). ❷ *Error Propagation Issue*: Compression of intermediate computing results spans multiple MoE layers, causing information distortion to accumulate layer-by-layer during MoE training, significantly compromising 18.9%-66.7% training accuracy (as see Figure 2 (b)). ❸ *Communication Imbalance Issue*: existing communication compression techniques address all data uniformly without considering the imbalanced communication load among experts, thereby ineffective in resolving the dilemma of low resource utilization and congested all-to-all communication (as see Figure 2 (c)).

In this paper, we propose a novel load-aware **Bi-random** quantization technique tailored for **MoE** training system, named BirdMoE, to solve the aforementioned issue based on several observations and techniques as below.

To solve the **Overhead Amplification Issue**, we leverage the lightweight advantage of the random process and introduce a Probabilistic Thresholding-based Random Quantization (RQ) for MoE layer, which significantly reduces compression overheads by probabilistically mapping the floating-point intermediate computing results of the all-to-all communication into low-bit integers at a low $O(N)$ time complexity.

To solve the **Error Propagation Issue**, we exploit the convergence efficiency of unbiased representation [18], [19] and build the expectation invariance within RQ, which can preserve an unbiased estimation for the quantized intermediate computing results, thereby effectively reducing the information distortion and maintaining the MoE training quality.

To solve the **Communication Imbalance Issue**, we study the specific data routing characteristic of MoE layer and introduce a Mixed Precision (MP) strategy upon the above RQ, which can dynamically adjust the bit-widths of data routed to different experts, effectively balancing the communication load and improve efficiency to the overall MoE training system.

To the best of our knowledge, BirdMoE is the first compression technique designed for balancing and reducing the communication costs of intermediate computing results in all-to-all communication while maintaining model accuracy in the MoE training context. The right bars of Figure 1 indicate that BirdMoE successfully reduces communication costs at low overheads, making MoE training accessible in platforms with limited computing and communication resources. Our main contributions are summarized as follows:

- We identify three issues when applying communication compression techniques to MoE training: (*i*) Compressing the intensive communications incurs significant computational overhead; (*ii*) The compression error of intermediate results severely degrades the MoE training quality; (*iii*) Compression cannot address the low training efficiency caused by imbalanced communication loads.
- We propose BirdMoE, a novel load-aware MoE communication compression technique with two key modules: *Mixed Presicion* and *Random Quantization*, to dynamically balance and reduce all-to-all communication costs at low $O(N)$ time complexity. Additionally, we establish expectation invariance within BirdMoE to ensure the unbiasedness of the compressed intermediate computing results, effectively maintaining MoE training quality.
- We conduct comprehensive evaluations on mainstream MoE training tasks, involving multiple model structures and datasets at different scales. The evaluation demonstrate that compared with the state-of-the-art approaches, BirdMoE can achieve higher 4.06×-10.44× communication compression ratios with 1.18×-5.27× training speedup while maintaining the MoE training quality.

II. Background and Motivation

A. MoE Structure and observed Communication Bottleneck

MoE Structure: MoE is found to have a strong ability in modern large-scale pre-trained models. The key idea of MoE is to constitute numerous small sub-models, namely experts, to a large model, given the intuition that different small models are experts in different domains, and can be only activated when the data in its domain are input. The most common usage of MoE is to extend the MLP layer of the state-of-the-art sequences Transformer [1], [21], [22] by a large number of Feed Forward Network (FFN) (i.e., experts) [7], [11]. For each training iteration, samples are fed into a few experts, dynamically selected by a trainable gate network. because experts are sparsely activated, much computation is saved. Hence MoE can significantly increase the number of samples trained in the same period and improve the model performance compared with conventional dense models [7], [23].

Data Flow: As introduced in Section I, the MoE layer are shared across multiple worker devices (e.g., GPU), while all other non-MoE layers are replicated. To route the data (e.g., *tokens*) among expert nodes, the '*all-to-all*' collective communication is required to perform four times communication between MoE and non-MoE layers per training iteration. Specifically, during the Feed-Forward (FF) propagation, two all-to-all communications are employed to '*dispatch*' and '*combine*' the *feature activation* between source devices of data and target device of desired experts, according to the results of the *gate network*. In the Back-Propagation (BP) propagation, two reverse all-to-all communication are used to transfer the *feature gradient* of corresponding feature activation along the reverse trace. As mentioned before, these intensive and imbalanced communications dominate the overall runtime of MoE training, significantly degrading MoE training efficiency.

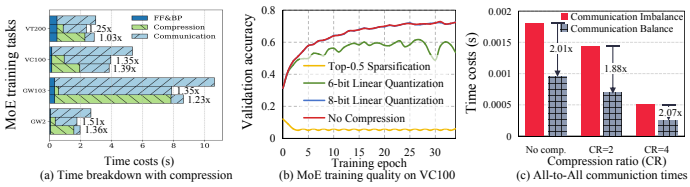

Fig. 2. Challenges of MoE communication compression. (a) Time breakdown of no-compression MoE training (top bar), linear quantization (middle bar), and Top-K sparsification (bottom bar). (b) Popular compression techniques degrade the model training quality to different degrees. (c) Compression with different levels cannot solve the communication imbalance issue.

B. Observed Compression Challenges in MoE

To examine the *Overhead Amplification* and *Error Propagation* issues discussed in Section I, we conduct various MoE training tasks on a distributed cloud-computing platform with 16 GPUs (detailed in Section IV-A). Then we apply two common communication compression techniques in distributed learning: Top-K sparsification and linear quantization, to reduce the communication costs of all-to-all stages during the MoE training.

Overhead Amplification issue: As illustrated in Figure 2 (a), time breakdown bars from up to bottom within a group represent the communication overheads of MoE training with no-compression, 8-bit linear quantization, and Top-0.1 sparsification. Experimental results show that sparsification reduces communication time by $63.5\% - 92.2\%$, but its naive implementation with $O(N \log K)$ complexity [24], [25] incurs significant compression overhead, yielding only a modest $1.03 \times - 1.39\times$ overall training speedup. Although 8-bit linear quantization has low computational overhead, it achieves a limited $4\times$ compression ratio, resulting in limited $1.25 \times - 1.51\times$ overall training speedup.

Error Propagation issue: Figure 2 (b) shows that even setting a high 0.5 sparsity for Top-K sparsification (Top-0.5), the MoE training collapse occurred, resulting in the model failing to converge. On the other hand, while low bit-width linear quantization (e.g., 6-bit) exhibits instability in training and reduced accuracy, sufficiently high bit-width (e.g., 8-bit) can ensure the MoE accuracy comparable to the no-compression counterpart (e.g., yet low compression ratio). This is because excessive compression error in intermediate computing results flow through the network, causing information distortion of feed and backward propagation.

Communication Imbalance issue: Popular expert nodes accept more data requiring longer communication time, thereby forcing other nodes to wait for synchronization and increasing the overall communication time. Figure 2 (c) shows the communication times with and without quantization in balanced and imbalanced expert communication loads. We observe that the ratio of balanced and imbalanced communication runtime remains similar under different compression ratios, indicating that evenly compress to all data does not alleviate the congestion in the imbalanced MoE communication.

C. Motivation of Our Solution

Driven by the above observations, we aim to solve the aforementioned issues by developing a load-aware communication compression technique, BirdMoE, tailored for the imbalanced and intensive all-to-all communication in MoE training based on the following motivations: ① Motivated by the lightweight computational advantages of random process and quantization operation, we introduce a lightweight random quantization algorithm to compress all-to-all communication by mapping the intermediate results to the closer ends of the quantization interval at a larger probability; ② Motivated by the convergence efficiency of unbiased gradient compression techniques in data parallelism context, we establish the expectation invariance within the above random quantization algorithm by maintaining the unbiasedness for the quantized intermediate computing results; ③ Motivation by the compression ratio of quantization is determined by the bit-width, we construct a mixed precision mechanism upon the above random quantization, which effectively improves the communication efficiency of the overall MoE training system by dynamically assigning different quantization bit-widths to the data routed to different expert nodes. The detailed methodology of BirdMoE are detailed in the next Section III.

III. Design and Implementation

A. Overview

In this paper, we propose BirdMoE, a load-aware Bi-random quantization technique for the MoE training with two key modules: *Mixed Precision* and *Random Quantization*.

1) *Mixed Precision (MP):* To balance the communication loads among expert nodes: (i) MP establishes a probabilistic negative correlation between bit-width and quantity of data routed to each expert. (ii) MP randomly assigns specific bit-widths to data routed to various experts according to a constructed probability distribution.
2) *Random Quantization (RQ):* For the intermediate computing results of experts in all-to-all communication: (i) RQ lowers compression overhead by randomly mapping the intermediate computing results to the closer integers at a larger probability. (ii) RQ maintains convergence efficiency of MoE training by building unbiasedness within the quantized intermediate computing results.

The detailed design and implementation of MP and RQ are introduced in the next subsections.

B. Mixed Precision (MP)

To balance the communication load in MoE training, MP assigns various bit-widths for the data routed to different expert nodes. The insight is that the quantization bit-width assigned to the data routed to a particular expert should be negatively proportional to the quantity.

Bit-width Assignment: For any expert node, assuming that a pre-set fixed bit-width is L (as in traditional fixed-precision quantization methods), the data quantity and bit-width routed to i-th node are N_i and L_i, respectively. As illustrated in

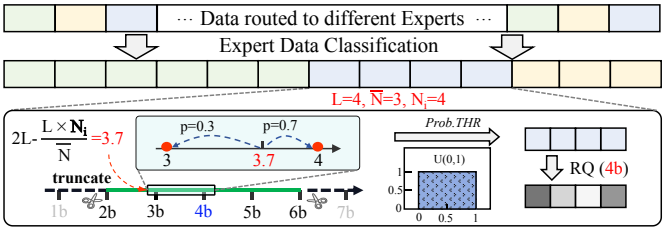

Fig. 3. A general example of Mixed Precision module.

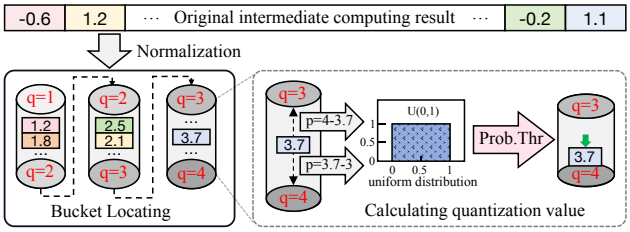

Fig. 4. A general example of Random Quantization module.

Figure 3, we first construct a bit-width intervals by expanding L to a bit-width range with multiple intervals as $\widetilde{L} \in \{1, 2, ..., L, ..., 2L-1\}$, centered at L. After that, MP randomly assigns L_i to the item of $\widetilde{L}$ by the following five steps: ① Calculating a scaler $s = \frac{N_i}{\overline{N}}$ to represent the factor of N_i relative to $\overline{N}$, where $\overline{N}$ is the average data quantity routed to all expert nodes. ② Determining the located bit-width interval $[b, b+1]$ within $\widetilde{L}$ according to s, where $b = 2L - floor(L \times s)$ is the left endpoint of the interval and $b+1$ is the right endpoint. As a result, a negative correlation is built between the data quantity routed to a specific expert node and the corresponding assigned bit-width. ③ Calculating the distance from $2L - L \times s$ to b and $b+1$ as $d_l = (b+1) - (2L - L \times s)$ and $d_{l+1} = (2L - L \times s) - b$, respectively, where $2L - L \times s$ is the location of N_i in the interval $[b, b+1]$. ④ Due to both d_l and d_{l+1} belonging to $[0, 1]$ while summing to 1, we can set the probability of assigning L_i to b and $b+1$ as the d_l and d_{l+1}, respectively. In doing so, L_i will be assigned to the closer endpoint (e.g., b or $b+1$) at a larger probability. ⑤ Assigning L_i to the specific endpoint as the bit-width by comparing the probability with a random number generated from the uniform distribution (i.e., *Probabilistic Thresholding (Prob. THR)* method) at a low computational cost. Notably, it can be derived that the average expectation of bit-width $\mathbb{E}[L_i] = L$ with $\forall N_i \leq 2\overline{N}$. Therefore, although data transferred to different nodes are randomly assigned with different bit-width, the average expectation of bit-width could be maintained at L.

Interval Truncating: In an extremely imbalance situation, data routed to one expert node may be very huge that $\exists N_i > 2\overline{N}$ (e.g., $N_1 = 3, N_2 = 5, N_3 = 1000$), thus the corresponding bit-width will be assigned to 1. While excessively low 1-bit quantization significantly reduces communication costs, it potentially degrades MoE training quality. To balance the communication costs and quantization error, MP bounds the lowest and highest bit-widths by truncating $\widetilde{L}$ as $\{L-\delta, ..., L-1, L, L+1, ..., L+\delta\}$. In doing so, we can flexibly balance communication costs and MoE accuracy while maintaining the average bit-width approximately equal to L. Note that the mixed precision strategy is degraded to the conventional fixed precision quantization if δ is set to 0.

Essentially, the Mixed Precision strategy balances the communication load at the cost of reducing the quantization precision of more data routed to several experts, thus there is a trade-off between δ values and MoE training quality. By evaluating multiple mainstream MoE training tasks, we notice that when δ is set to 2, the all-to-all communication congestion could be effectively eliminated without significantly compromising model training accuracy. After assigning the specific bit-width for the data routed to each expert, BirdMoE utilizes next RQ module to randomly quantize the original floating-point intermediate computing results to integers as follows.

C. Random Quantization (RQ)

As introduced in subsection II-B, lossy compression techniques potentially incur *Overhead Amplification* and *Error Propagation* issues. By exploiting the lightweight advantage of the random process and the convergence efficiency of the expectation invariance, we propose an unbiased Random Quantization (RQ) algorithm to efficiently compress the intermediate computing results of all-to-all communication in MoE layers. As illustrated in Figure 4, the insight of random quantization is to map the floating-point into the closer integer end of the quantization interval with a larger probability.

All-to-all Compression: Given the original floating-point intermediate computing result set $\mathcal{X}$ and the assigned bit-width L, the quantization counterpart of $\mathcal{X}$ is denoted by $\mathcal{Q}$. There is a natural range of quantization level $[1, 2, ..., L]$. A similar methodology as MP is employed to map the i-th elements x_i from $\mathcal{X}$ into $\mathcal{Q}$ as below: ① Calculating the normalized floating points as $f_i = \frac{|x_i|}{||\mathcal{X}||_\infty}$, which is used to reflect the proportion of x_i within the entire range of $\mathcal{X}$; ② Locating the bucket $[l, l+1]$ for x_i according to f_i, where $l = floor(f_i \times L)$ is the left endpoint of the bucket and $l+1$ is the right endpoint. ③ Calculating the distance from f_i to each endpoint as $d_l = f_i \times L - l$ and $d_{l+1} = l + 1 - f_i \times L$, where d_l and d_{l+1} are located in $[0, 1]$ while summing to 1. ④ Establishing a negative correlation between distance and probability by setting the probability of mapping x_i to $l+1$ and l as d_l and d_{l+1}, respectively. In doing so, x_i will be mapped to the closer endpoint (e.g., l or $l+1$) at a larger probability. ⑤ Quantizing x_i to the specific bucket endpoint (e.g., l or $l+1$) as the quantized value q_i using the *Probabilistic Thresholding* method with a low $O(N)$ computational complexity.

Unbiased Recovery: After the above random quantization, the sender node only needs to transfer (*i*) the signed quantized integer q_i, each element of $L+1$-bit; (*ii*) one floating-point factor $||\mathcal{X}||_\infty$, to the target expert node. The receiver node can recover the unbiased and trainable floating-point data with $\mathbb{E}[(\mathcal{Q})] = \mathcal{X}$. The convergence efficiency of similar algorithms has been theoretically proven in gradient compression of data parallelism scenario. Empirically evaluations suggest that RQ

module can effectively mitigate the *Error Propagation* issue in the compression of intermediate computing results using the recommended 3-5 bit-widths, thereby effectively maintaining the MoE training quality (detailed in subsection IV-B).

IV. Experiments

A. Experimental Setup

Tasks, Models, and Datasets: Our experimental workloads cover four typical MoE training tasks, which are trained on various models and datasets at different scales: ② GW2: Training a GPT-based MoE [22] model with 19.2 million parameters on WikiText2 [26] dataset using the Adam optimizer with a learning rate of 1e-2. ③ GW103: Training a larger GPT-based MoE model with 315.1 million parameters on WikiText103 [26] dataset using the Adam optimizer with a learning rate of 1e-2. ④ VC100: Training a ViT-based MoE model [23] with 19.0 million parameters on CIFAR100 [27] dataset using the Adam [28] optimizer with a learning rate of 1e-3. ⑤ VT200: Training a larger ViT-based MoE model with 181.3 million parameters on Tiny ImageNet [29] dataset using the Adam optimizer with a learning rate of 1e-3.

Baseline and compared methods: To demonstrate the superiority of our proposed BirdMoE, we select three popular state-of-the-art quantization-based communication compression techniques: QSGD [14], PWLQ [30], and ShapeShifter [31] to compare the MoE training quality, compression ratio, compression throughput, and training speedup across the above four MoE training tasks. Additionally, we also provide the performance of the original no-compression baseline to refer.

Experimental environment: We implement the MoE training environment upon a distributed cloud-computing platform with 16 GPUs within 4 nodes, and these nodes are connected by a limited 10 Gbps inter-node Ethernet. Within each node, 4 NVIDIA Tesla V100-SXM2-16GB GPUs are connected by a 400 Gbps intra-node NVLink. The compression operations of different methods are implemented using the `Pytorch v.2.2.1` [32] package in Python 3.8.13. The all-to-all communication is built on the collective communication of `torch.distributed` [32] package, utilizing the `NCCL v.2.19.3` backend.

B. Evaluation of MoE Training Quality

The most important premise in communication compression is to ensure that the model training quality remains unaffected significantly. Thus we first explore the superiority of BirdMoE in maintaining MoE training quality (i.e., model accuracy and convergence rate). To study the impact of our proposed mixed precision module on model training quality, we provide the evaluation results of BirdMoE with mixed and fixed precision (referred to as $\text{BirdMoE}_{\text{MP}}$ and $\text{BirdMoE}_{\text{FP}}$). For the sake of fairness, we first compare BirdMoE with the state-of-the-art quantization-based compression approaches QSGD, PWLQ, and ShapeShifter under the same bit-width as BirdMoE.

Figure 5 shows that when we assign different compression methods to the same bit-width, BirdMoE with fixed precision (i.e., $\text{BirdMoE}_{\text{FP}}$) achieves the best MoE training quality, which has a similar performance to the original no-compression MoE training solution. Specifically, ShapeShifter combines a global linear quantization with a lossless encoding to compress the flatten intermediate computing results without considering the distribution difference among data dimension, thereby failing to maintain the convergence stability of MoE training and degrading the model accuracy in GW103 and VT200 tasks. Although QSGD can compress gradients while maintaining the convergence efficiency in the data parallelism by preserving the expectation invariance of L_2-norm, the same measurement is ineffective for quantizing the intermediate results compression, thus causing a significant MoE training divergence. PWLQ employs a piece-wise quantization strategy to reduce the information distortion of intermediate computing results, effectively maintaining the MoE model training quality in GW2 and VC100 tasks. However, PWLQ cannot achieve a stable convergency in the other tasks with the larger model capacity due to its disregard for the distribution differences in data routed to different MoE layer. In contrast, BirdMoE with a Mixed Precision strategy (i.e., $\text{BirdMoE}_{\text{MP}}$) reduces the quantization precision of more data, slightly degrading model accuracy of $\text{BirdMoE}_{\text{FP}}$, while its craft unbiasedness enables it to achieve comparable and stable MoE training quality with further improving all-to-all communication efficiency.

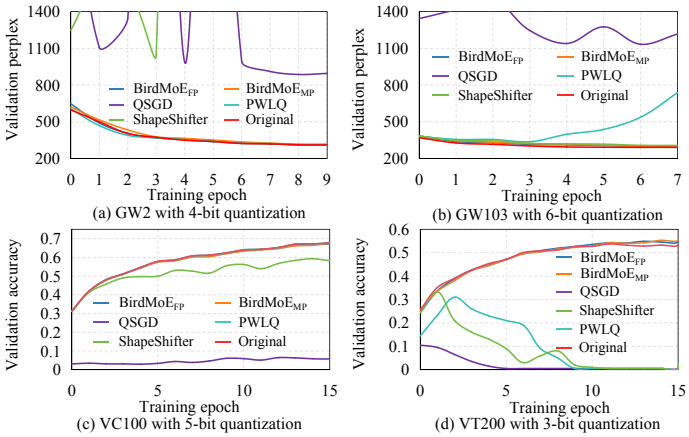

Fig. 5. $\text{BirdMoE}_{\text{FP}}$ and $\text{BirdMoE}_{\text{MP}}$ achieve better MoE training quality than other methods under the same bit-width.

C. Evaluation of Compression Ratio

Figure 6 (a) shows the communication compression ratio of different methods, calculated by the original communication costs of the no-compression solution divided by the compressed communication costs of each method under their **best practice**. Note that the best practice refers to the maximum compression ratio a particular method can achieve while maintaining the similar original MoE accuracy, given the same number of MoE communication rounds as the no-compression solution. Evaluation results indicate that $\text{BirdMoE}_{\text{FP}}$ achieves 5.3×-7.4× compression ratios in vision tasks, outperforming competing methods, while PWLQ and ShapeShifter have a comparable 2.2×-7.2× compression ratio in language tasks. This is because the compressibility of intermediate computing

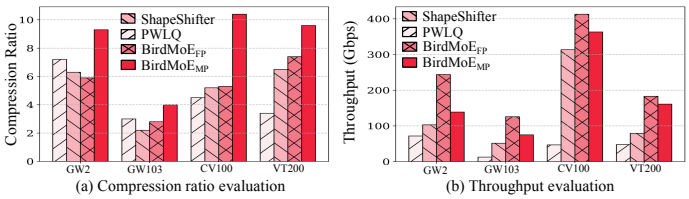

Fig. 6. The evaluation of communication compression ratio and compression throughput (Gbps) across various tasks.

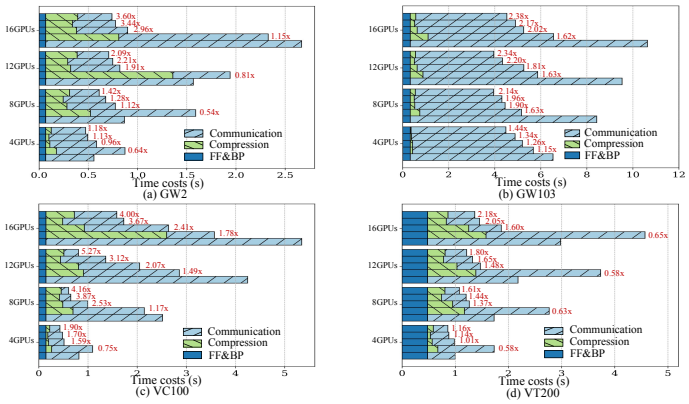

Fig. 7. The average time breakdowns per iteration on 4, 8, 12, and 16 GPU nodes. Bars from top to bottom are time breakdowns of BirdMoE$_{MP}$, BirdMoE$_{FP}$, ShaperShfter, PWLQ, and no-compression.

results differs among the various model structures, leading to different compression performances across tasks. In contrast, BirdMoE$_{MP}$ achieves superior 4.1×-10.4× compression ratios across all tasks by further reducing the bit-width of intermediate computing results with large quantities, thereby significantly increasing the overall compression ratio.

D. Evaluation of Throughput

Figure 6 (b) illustrates the compression throughput of different techniques, which is defined as $tp = \frac{CS}{CT}$, where CS is the amount of reduced data volume by compression, CT is the compression runtime. Compression throughput can effectively reflect the overall performance of an approach by comprehensively considering the compression overheads and gains. Experimental results suggest that although PWLQ and Shapeshifter can achieve a slightly higher compression ratio in partial tasks, their piece-wise calculation and lossless coding incur significant computational overheads, thereby limiting their compression throughput to 12.6-313.5 Gbps. In contrast, BirdMoE$_{FP}$ achieves the highest 125.7-612.8 Gbps compression throughput by employing lightweight random quantization to efficiently achieve a comparable compression ratio. On the other hand, BirdMoE$_{MP}$ utilizes MP module to further reduce the communication costs with an acceptable extra computational cost, achieving the second-best compression 75.1-363.2 Gbps compression throughput.

E. Scalability and Time Breakdown Evaluation

The essential purpose of communication compression is to speed up MoE training at different scales of nodes, thus we further deploy BirdMoE upon the MoE training environment containing various numbers of 4, 8, 12, and 16 GPU nodes. Then we validate our scalability superiority by comparing the average time breakdown and MoE training speedup performance with different compression methods.

Figure 7 illustrates the average time breakdown per iteration of the runtimes of forward and backward propagation (FF&BP), communication, and compression. The red values are acceleration rate (the runtime of the original no-compression MoE training divided by the runtime of MoE training with compression). Experimental results suggest that (*i*) the moderate communication compression ratio and significant compression overheads of PWLQ and Shapeshifter limit their training speedup performance and even prolong the training time to several tasks. Additionally, their inability to address the imbalanced communication load also limits the training speedup performance. (*ii*) BirdMoE$_{FP}$ achieves comparable compression ratio on different tasks at low computational costs, obtaining higher MoE training speedup of 1.13×-3.67×. (*iii*) BirdMoE$_{MP}$ further reduces the communication time by balancing the communication traffic among GPUs, obtaining the highest 1.18×-5.27× MoE training speedup. (*iv*) BirdMoE's training speedup performance improves with the increase of nodes. That is because increasing the number of nodes will increase the compressed communication volume without increasing the compression overhead due to the BirdMoE is deployed on different nodes and executed in parallel. Generally, the above results demonstrate that BirdMoE efficiently reduces communication costs and improves the efficiency of MoE training with different numbers of nodes. Therefore, the good scalability of BirdMoE not only advances research on MoE for resource-constrained platforms but also scales effectively to large-scale MoE training environments.

V. Conclusion and Future Work

In this paper, we propose a lightweight load-aware communication compression technique BirdMoE, which utilizes random quantization and mixed precision to compress the imbalanced and intensive communication of MoE training. Compared with the state-of-the-art approaches, experiments on mainstream MoE training tasks demonstrate that BirdMoE achieves higher 4.06×-10.44× communication compression ratio and 1.18×-5.27× training speedup while maintaining the MoE training quality. In the future, we aim to optimize the efficiency of BirdMoE with the CUDA and implement it into the all-to-all collaborative communication library.

Acknowledgment

This work was supported in part by the Guangdong S&T Program under Grant 2024B0101010002, in part by the National Natural Science Foundation of China (Nos. U2436208, 62372129, 62472127, 62032023, and T2125013), in part by the Project of Guangdong Key Laboratory of Industrial Control System Security under Grant 2024B1212020010, in part by the Innovation Funding of ICT, CAS under Grant No. E461050. (Corresponding author: Zhihong Tian; Wen Xia).

REFERENCES

[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, and et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR'21*, 2021.

[2] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT'19*, 2019, pp. 4171–4186.

[3] D. Wu, W. Yang, H. Jin, and et al., "Fedcomp: A federated learning compression framework for resource-constrained edge computing devices," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst*, vol. 43, no. 1, pp. 230–243, 2024.

[4] R. Zhang, M. Javaheripi, Z. Ghodsi, and et al., "Adagl: Adaptive learning for agile distributed training of gigantic gnns," in *proc. DAC'23*, 2023.

[5] D. Wu, X. Zou, S. Zhang, and et al, "Smartidx: Reducing communication cost in federated learning by exploiting the cnns structures," in *Proc. AAAI'22*, 2022, pp. 4254–4262.

[6] D. Wu, W. Yang, X. Zou, and et al., "Smart-dnn+: a memory-efficient neural networks compression framework for the model inference," *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 4, pp. 1–24, 2023.

[7] N. Shazeer, A. Mirhoseini, K. Maziarz, and et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR 2017*, 2017.

[8] J. He, J. Zhai, T. Antunes, and et al, "Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models," in *Proc. PPoPP '22*, 2022.

[9] J. Li, Y. Jiang, Y. Zhu, and et al., "Accelerating distributed moe training and inference with lina," in *Proc. USENIX ATC 2023*, 2023.

[10] M. Zhai, J. He, Z. Ma, and et al., "Smartmoe: Efficiently training sparsely-activated models through combining offline and online parallelization," in *Proc. USENIX ATC 2023*, 2023.

[11] D. Lepikhin, H. Lee, Y. Xu, and et al., "Gshard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR 2021*, 2021.

[12] F. Xue, Z. Shi, F. Wei, and et al., "Go wider instead of deeper," in *proc. AAAI'22*, 2022, pp. 8779–8787.

[13] D. Wu, W. Yang, X. Zou, and et al., "Bird+: Design of a lightweight communication compressor for resource-constrained distribution learning platforms," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 11, pp. 2193–2207, 2024.

[14] D. Alistarh, D. Grubic, J. Li, and et al., "QSGD: communication-efficient SGD via gradient quantization and encoding," in *Proc. NIPS'17*, 2017.

[15] D. Wu, W. Yang, C. Deng, and et al., "Bird: A lightweight and adaptive compressor for communication-efficient distributed learning using tensor-wise bi-random sampling," in *proc. ICCD'23*. IEEE, 2023.

[16] Y. Lin, S. Han, H. Mao, and et al., "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *Proc. ICLR'18*, 2018.

[17] T. Vogels, S. P. Karimireddy, and M. Jaggi, "Powersgd: Practical low-rank gradient compression for distributed optimization," in *Proc. NIPS'19*, 2019, pp. 14 236–14 245.

[18] Y. He, X. Huang, and K. Yuan, "Unbiased compression saves communication in distributed optimization: When and how much?" in *Proc. NIPS'23*, 2023.

[19] L. Condat, K. Yi, and P. Richtárik, "EF-BV: A unified theory of error feedback and variance reduction mechanisms for biased and unbiased compression in distributed optimization," in *Proc. NIPS'22*, 2022.

[20] H. Xu, K. Kostopoulou, A. Dutta, and et al., "Deepreduce: A sparse-tensor communication framework for federated deep learning," *in Proc. NIPS'21*, pp. 21 150–21 163, 2021.

[21] A. Vaswani, N. Shazeer, N. Parmar, and et al., "Attention is all you need," in *Proc. NIPS'17*, 2017, pp. 5998–6008.

[22] T. B. Brown, B. Mann, N. Ryder, and et al., "Language models are few-shot learners," in *Proc. NIPS'20*, 2020.

[23] C. Riquelme, J. Puigcerver, B. Mustafa, and et al., "Scaling vision with sparse mixture of experts," in *Proc. NIPS'21*, 2021.

[24] S. Shi, X. Zhou, S. Song, and et al., "Towards scalable distributed training of deep learning on public cloud clusters," in *Proc. MLSys'21*, 2021.

[25] A. Mandal, H. Jiang, A. Shrivastava, and V. Sarkar, "Topkapi: Parallel and fast sketches for finding top-k frequent elements," in *Proc. NIPS'18*, 2018, pp. 10 921–10 931.

[26] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *Proc. ICLR'17*, 2017.

[27] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Tech. Rep., 2009.

[28] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR'15*, 2015.

[29] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. NIPS'12*, pp. 1106–1114.

[30] J. Fang, A. Shafiee, H. Abdel-Aziz, and et al., "Post-training piecewise linear quantization for deep neural networks," in *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part II*, ser. Lecture Notes in Computer Science, vol. 12347. Springer, 2020, pp. 69–86.

[31] A. D. Lascorz, S. Sharify, I. E. Vivancos, and et al., "Shapeshifter: Enabling fine-grain data width adaptation in deep learning," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2019, Columbus, OH, USA, October 12-16, 2019*. ACM, 2019, pp. 28–41.

[32] A. Paszke, S. Gross, F. Massa, and et al., "Pytorch: An imperative style, high-performance deep learning library," in *Proc. NIPS'19*, 2019.